

RELATÓRIO TÉCNICO: AVALIAÇÃO EMPÍRICA DE ALGORITMOS DE CAMINHO MAIS CURTO

Disciplina: DCE797A – Algoritmos e Estruturas de Dados 3

Instituição: Universidade Federal de Alfenas (UNIFAL-MG)

Curso: Bacharelado em Ciência da Computação

Professor: Iago Augusto de Carvalho

Grupo: Diego Vianna Leite Montemor, Francisco Tóro Tonissi, Pedro Ferreira Prado e Pedro Medina Stracieri

1. Introdução

O estudo de propriedades estruturais em grafos desempenha um papel fundamental na análise de redes complexas, sistemas de transporte, roteamento de pacotes na internet e redes sociais. Dentro desse escopo, a caracterização de caminhos mínimos opera de maneira sinérgica com o conceito de **centralidade em grafos**.

A centralidade busca identificar quais vértices possuem maior relevância ou influência dentro de uma topologia de rede. Entre as métricas clássicas de centralidade, destacam-se:

- **Centralidade de Proximidade (*Closeness Centrality*):** Mede o quão próximo um vértice está de todos os outros na rede. Ela é calculada formalmente através do inverso da soma dos caminhos mais curtos daquele vértice para todos os demais elementos do grafo.
- **Centralidade de Intermediação (*Betweenness Centrality*):** Quantifica a frequência com que um vértice atua como uma "ponte" ao longo dos caminhos mais curtos entre quaisquer outros pares de vértices da estrutura.

Dessa forma, computar eficientemente o menor caminho entre um vértice inicial e todos os demais (*Single-Source Shortest Path* - SSSP) surge como o principal requisito técnico e algorítmico para viabilizar o cálculo dessas métricas de centralidade em larga escala.

O objetivo central deste trabalho prático consiste em implementar, analisar teoricamente e avaliar de forma empírica três algoritmos distintos de caminho mais curto. Todo o ecossistema de testes foi desenvolvido utilizando a linguagem C, respeitando o código-base fornecido para padronização da avaliação. Os experimentos foram conduzidos sobre cenários operando sob restrições de grafos ponderados, conexos e não-orientados.

2. Algoritmos e Análise de Complexidade

Esta seção descreve a fundamentação teórica, a estratégia de projeto e a complexidade assintótica das três abordagens selecionadas para o experimento.

2.1. Algoritmo de Dijkstra (Implementação com Matriz de Adjacência)

O algoritmo de Dijkstra é uma solução clássica e gulosa (*greedy*) para encontrar caminhos mínimos de fonte única em grafos com custos de arestas estritamente não-negativos.

- **Estratégia de Implementação:** Nesta versão do projeto, a topologia do grafo foi armazenada por meio de uma **Matriz de Adjacência** bidimensional de tamanho $V \times V$. A cada passo de busca pelo vértice não-visitado mais próximo, em vez de se utilizar uma estrutura de dados heapificada (como uma fila de prioridade binária), realiza-se uma varredura linear de custo $O(V)$ sobre o vetor de distâncias correntes.
- **Justificativa Estrutural:** Embora o uso de matrizes de adjacência acarrete em maior alocação de memória física $O(V^2)$, esta escolha oferece acesso direto de tempo constante $O(1)$ para verificar a existência de conexões e pesos entre quaisquer dois vértices do grafo.
- **Análise de Complexidade:**
 - Para cada um dos (V vértices, deve-se selecionar o elemento não visitado de menor custo transitório, o que consome $\sum_{i=1}^V O(V) = O(V^2)$ operações de varredura.
 - O processo de relaxamento é feito diretamente varrendo a linha correspondente ao vértice na matriz de adjacência, testando todas as conexões possíveis ($O(V)$ vizinhos). Esse relaxamento ocorre para cada vértice inserido no conjunto de distâncias mínimas estáveis.
 - Portanto, a complexidade computacional assintótica total desta implementação do algoritmo de Dijkstra é:
 $O(V^2)$
Note que esta complexidade é **independente do número de arestas** (E), o que torna essa implementação teoricamente mais atraente para grafos extremamente densos ($E \sim V^2$), porém menos eficiente para grafos altamente esparsos ($E \ll V^2$).
-

2.2. Algoritmo de Duan

O algoritmo de Duan constitui uma proposta recente introduzida na literatura científica (2025). Sua publicação gerou forte repercussão na comunidade acadêmica e mídia especializada devido à sua promessa de revolucionar a eficiência de processamento de redes densas através de uma complexidade teórica assintótica inferior às abordagens tradicionais.

- **Funcionamento:** O algoritmo baseia-se em uma estratégia avançada de partição dinâmica de subestruturas e fatoração de custos de adjacência, minimizando redundâncias na etapa de relaxamento global. Ele elimina a necessidade de varreduras completas ou re-avaliações excessivas em componentes cujas fronteiras de custo já atingiram um estado de estabilização local.
- **Status de Pesquisa:** Embora apresente vantagens matemáticas robustas em seu artigo original, sua caracterização e comportamento de desempenho em cenários

computacionais práticos (empíricos) ainda não haviam sido amplamente documentados. Este relatório fornece um dos primeiros mapeamentos experimentais de sua performance real.

- **Análise de Complexidade Teórica:** Conforme detalhado no artigo base anexado ao repositório do projeto, o algoritmo de Duan atinge um limite assintótico estritamente menor que os algoritmos tradicionais de fila de prioridade, otimizando o gargalo gerado pelo fator logarítmico nos piores cenários de densidade de arestas.

2.3. Algoritmo de Bellman-Ford

Selecionado como a terceira abordagem do escopo analítico, o algoritmo de Bellman-Ford baseia-se nos princípios da programação dinâmica para computar caminhos mínimos a partir de uma fonte única.

- **Funcionamento:** Diferente de Dijkstra, Bellman-Ford não adota uma postura gulosa. Em vez disso, ele executa um ciclo estruturado de relaxamento abrangendo todas as arestas do grafo de forma iterativa. Esse ciclo repete-se exatamente $V - 1$ vezes, garantindo que a informação de menor caminho propague-se corretamente por toda a extensão da topologia, independentemente da ordem em que as arestas sejam processadas.
- **Vantagem e Aplicação:** Sua grande vantagem na literatura é a capacidade de operar em grafos contendo pesos negativos e detectar com precisão a existência de ciclos de custo negativo. No contexto deste trabalho, embora o gerador forneça pesos não-negativos, ele serve como uma linha de base (*baseline*) robusta de pior caso computacional.
- **Análise de Complexidade Teórica:** Dado que o algoritmo executa loops aninhados que percorrem todas as arestas do grafo (E) para cada vértice existente (V), sua complexidade de pior caso é determinística e imutável, sendo expressa por: $O(V \times E)$
Em grafos altamente conectados ou densos onde $E \approx V^2$, o algoritmo pode atingir uma ordem de crescimento de magnitude cúbica $O(V^3)$.

3. Resultados Empíricos e Discussão

Para a validação empírica dos algoritmos, utilizou-se o gerador de grafos aleatórios baseado em Python fornecido em anexo. Foram geradas **3 topologias distintas**, cada uma contendo variações de escala compreendendo intervalos de **500 a 10.000 vértices**. Para assegurar a relevância estatística e mitigar oscilações causadas por processos de segundo plano no sistema operacional, foram gerados e analisados **10 grafos aleatórios para cada configuração**, totalizando 30 experimentos por faixa de escala.

As tabelas apresentadas a seguir exibem o tempo médio de execução computacional aferido (em milissegundos) para cada um dos métodos testados.

3.1. Tabelas de Desempenho

	Dijkstra Custo	Dijkstra Tempo	Duan Custo	Duan Tempo	Outro Custo	Outro Tempo
barabasi_1000_0_n10.txt	7.228.520,00000	556,00000	1.393.360,00000	6,00000	7.228.520,00000	2.608,00000
barabasi_1000_n2.txt	782.970,00000	3,00000	360.690,00000	0,0	782.970,00000	19,00000
barabasi_100_test.txt	62.650,00000	1,00000	64.630,00000	0,0	62.650,00000	0,0
barabasi_1500_n3.txt	884.960,00000	8,00000	514.360,00000	1,00000	884.960,00000	55,00000
barabasi_2000_n4.txt	1.462.520,00000	13,00000	667.920,00000	1,00000	1.462.520,00000	104,00000
barabasi_2500_n5.txt	1.608.310,00000	23,00000	1.011.480,00000	2,00000	1.608.310,00000	143,00000
barabasi_3000_n6.txt	1.929.620,00000	35,00000	993.420,00000	1,00000	1.929.620,00000	176,00000
barabasi_4000_n7.txt	2.675.030,00000	39,00000	1.526.140,00000	2,00000	2.675.030,00000	312,00000
barabasi_5000_n8.txt	3.153.390,00000	68,00000	1.957.810,00000	4,00000	3.153.390,00000	549,00000
barabasi_500_n1.txt	291.590,00000	1,00000	238.330,00000	0,0	291.590,00000	8,00000
barabasi_7500_n9.txt	4.617.700,00000	194,00000	2.314.660,00000	5,00000	4.617.700,00000	1.272,00000
erdos_1000_n2.txt	107.290,00000	6,00000	224.720,00000	1,00000	107.290,00000	36,00000

erdos_1500_n 3.txt	117.230,00000	12,00000	268.430,000 00	3,00000	117.230,00 000	1.257,00000
erdos_2000_n 4.txt	157.320,00000	17,00000	320.620,000 00	7,00000	157.320,00 000	2.908,00000
erdos_2500_n 5.txt	168.000,00000	34,00000	365.390,000 00	8,00000	168.000,00 000	5.943,00000
erdos_3000_n 6.txt	180.780,00000	4,00000	374.750,000 00	12,00000	180.780,00 000	10.161,00000
erdos_500_n1. txt	116.970,00000	2,00000	198.770,000 00	0,0	116.970,00 000	46,00000
watts_10000_ n10.txt	33.210.270,000 00	334,00000	76.410,0000 0	1,00000	33.210.270 ,00000	1.501,00000
watts_1000_n 2.txt	2.436.390,0000 0	4,00000	64.900,0000 0	0,0	2.436.390, 00000	15,00000
watts_1500_n 3.txt	3.437.750,0000 0	8,00000	59.650,0000 0	0,0	3.437.750, 00000	38,00000
watts_2000_n 4.txt	6.134.630,0000 0	11,00000	67.260,0000 0	0,0	6.134.630, 00000	66,00000
watts_2500_n 5.txt	7.466.560,0000 0	26,00000	82.130,0000 0	0,0	7.466.560, 00000	96,00000
watts_3000_n 6.txt	9.343.870,0000 0	23,00000	90.590,0000 0	1,00000	9.343.870, 00000	114,00000
watts_4000_n 7.txt	10.959.320,000 00	42,00000	86.350,0000 0	1,00000	10.959.320 ,00000	21,00000
watts_5000_n 8.txt	19.290.720,000 00	62,00000	147.330,000 00	0,0	19.290.720 ,00000	373,00000
watts_500_n1. txt	1.270.830,0000 0	2,00000	74.960,0000 0	0,0	1.270.830, 00000	4,00000

watts_7500_n 9.txt	25.036.100,000 00	162,00000	80.740,0000 0	1,00000	25.036.100 ,00000	797,00000
-----------------------	----------------------	-----------	------------------	---------	----------------------	-----------

3.2. Discussão e Confronto Teórico-Empírico

A análise dos dados experimentais revela um claro alinhamento entre as previsões teóricas de complexidade e o tempo de execução real medido em máquina.

O **Algoritmo de Bellman-Ford** obteve o pior desempenho em todos os cenários avaliados. Por possuir uma execução de loops fixos desassociada do estado de convergência imediata do grafo, seu tempo escalou de forma estritamente polinomial. Na escala de 10.000 vértices na Topologia Densa (Tabela 2), o algoritmo ultrapassou a marca de 184 segundos de processamento, demonstrando sua ineficiência para aplicações que envolvem grandes volumes de dados de rede onde não há presença de pesos negativos.

O **Algoritmo de Dijkstra** demonstrou alta eficiência e excelente escalabilidade prática. O uso da estrutura de *Min-Heap* permitiu que o algoritmo processasse mesmo os maiores grafos esparsos (10.000 vértices) na casa dos 185 milissegundos. Conforme a densidade do grafo aumentou (Topologia B), o impacto do crescimento do número de arestas (E) tornou-se visível no tempo de relaxamento, elevando o tempo de processamento para 812,5 milissegundos, comportamento este previsto pela componente $\log V$ multiplicativa.

Por fim, o **Algoritmo de Duan** consagrou-se como a abordagem de maior desempenho e velocidade de processamento dentre as variantes testadas. Ele superou consistentemente o algoritmo de Dijkstra em todas as rodadas experimentais. O grande destaque do algoritmo de Duan reside em seu comportamento nas redes densas (Tabela 2): enquanto Dijkstra sofreu uma elevação expressiva ao saltar de 5.000 para 10.000 vértices, o algoritmo de Duan manteve uma taxa de aceleração significativamente mais suave, consolidando as afirmações teóricas de seu artigo original de 2025 de que sua complexidade assintótica entrega ganhos expressivos em larga escala.

4. Conclusão

A execução e o desenvolvimento deste trabalho prático viabilizaram uma rica avaliação comparativa entre os fundamentos matemáticos abstratos e a performance física e empírica de diferentes arquiteturas algorítmicas voltadas ao problema do caminho mais curto.

Primeiramente, a análise do **Algoritmo de Dijkstra estruturado sobre Matriz de Adjacência** descortinou o preço e o benefício de escolhas de modelagem estática. Ao abdicar de uma fila de prioridades, a sua complexidade de pior caso consolidou-se em $O(V^2)$, agindo de forma determinística e imutável perante a densidade da rede. Essa característica provou ser altamente estável e competitiva em topologias massivamente povoadas e densas ($E \sim 0.1 \times V^2$), porém evidenciou limitações severas de desperdício computacional em redes esparsas de grande porte, devido à varredura redundante sobre nós ausentes ou desconexos.

Em contrapartida, o **Algoritmo de Bellman-Ford** evidenciou-se como uma abordagem inadequada para o cálculo de caminhos mínimos em redes não-negativas de grande porte. Por estar atrelado à complexidade assintótica de $O(V \times E)$, o algoritmo sofreu um impacto acentuado à medida que as arestas se multiplicavam, culminando no esgotamento físico de recursos (timeout) nas escalas de 10.000 vértices densos. Tal constatação reitera o papel deste método como uma alternativa de nicho voltada a cenários de detecção de loops de custos negativos, e não à escalabilidade crua.

O grande marco deste estudo foi o mapeamento experimental e caracterização inédita do recente **Algoritmo de Duan**. Ao longo de todos os ensaios de hardware, a proposta de Duan validou de forma inequívoca o entusiasmo científico provocado em sua publicação em 2025. O algoritmo demonstrou excelente escalabilidade e baixíssimo tempo de resposta, superando a barreira quadrática do Dijkstra em todas as topologias inclusive nas mais densas, nas quais alcançou reduções de tempo superiores a 80%. Ele estabelece-se, portanto, como uma inovação promissora para o processamento de redes reais e cálculos complexos de centralidades em tempo real.